Research Tasks & Exploration Plan

QUIC Server-Side Migration Project

June 20, 2026

Detailed Version *— Comprehensive descriptions, research questions, comparison tables, and sub-task breakdowns. Print and read.*

Current Status

We have a **fully working cross-machine QUIC server migration** implementation using modified Mozilla Neqo (Rust). Verified with Firefox over HTTP/3 on 4 physical machines (same LAN). **Anik:** 5 state transfer backends. **Fatima:** Docker/namespace setup with /tmp migration.

Research Focus (Per Professor)

- **QUIC Proxy (Pass-Through Handshake)** — Explore primary as proxy during handshake. Compare against baselines.
- **Precise Definition of Applications** — What are the real applications of QUIC migration? Server architecture, engineering perspective.

All Tasks at a Glance

#	Task	Type
1	Docker Migration + 4 Backend Comparison (/tmp, TCP, Redis, QUIC)	Engineering + Benchmarking
2	QUIC Proxy (Pass-Through Handshake) Exploration + Decryption Problem	Protocol Exploration
3	Applications of QUIC Migration (Use Cases, Server Types, Stream LB, Server Chaining)	Research + Exploration
4	Neqo Server & Server Architecture (Neqo Capabilities, nginx Comparison, Architecture Requirements)	Investigation + Analysis
5	Attack Surface vs. Benefits Tradeoff Analysis	Security Analysis

Task 1: Docker-Based Migration with 4 Backend Comparison

Goal: Convert Fatima's namespace-based implementation to Docker containers. Implement and benchmark 4 state transfer backends: /tmp (shared volume), TCP Push, Redis KV, and QUIC-based transfer. Measure latency, reliability, and migration success rate.

Background

Fatima has completed a working migration using network namespaces with /tmp (shared filesystem) as the state transfer mechanism. The migration state is ~445 bytes containing TLS secrets, connection IDs, packet numbers, and client address. The next step is to convert this to Docker containers and implement additional backends.

Sub-Tasks

- **1a. Convert Namespace to Docker:** Each server (primary, preferred) in its own container. Shared Docker network for QUIC traffic. Redis in a third container.
- **1b. /tmp Backend (Shared Volume):** Primary writes state to shared Docker volume. Preferred reads from same volume. Baseline — zero network overhead. Leaves disk forensic artifacts.
- **1c. TCP Push Backend:** Primary opens TCP to preferred, pushes 445 bytes. Already implemented on physical machines — port to Docker. Observable side-channel (TCP connection visible to observer). Lowest latency (~1ms).
- **1d. Redis KV Backend:** State in Redis (separate container). Primary SETs, preferred GETs. Decoupled — supports lazy retrieval. Best multi-server scalability.
- **1e. QUIC-Based State Transfer (NEW):** QUIC connection between primary and preferred for state transfer. Encrypted, indistinguishable from regular QUIC traffic. Hardest for IDS to detect. New implementation required.

Comparison Metrics

- Transfer latency (state export to preferred ready)
- Migration success rate (over 100 consecutive migrations)
- Time-to-first-byte after migration (client perspective)
- Network fingerprint visibility (tcpdump + Wireshark analysis)
- Disk/memory forensic artifacts
- Scalability (N preferred servers)
- Failure resilience (preferred not ready scenario)

Expected Comparison Matrix

Backend	Latency	Network Visible?	Forensic Artifacts	Scalability	Dependencies
/tmp (shared vol)	Lowest (disk I/O)	No	Yes (file on disk)	Single host only	Shared storage
TCP Push	~1ms (LAN)	Yes (TCP conn)	No (in-memory)	Point-to-point	None
Redis KV	~2-3ms (LAN)	Yes (Redis proto)	Yes (Redis log)	Multi-server	Redis instance

QUIC Transfer	~2-5ms (LAN)	Encrypted (looks normal)	No (in-memory)	Point-to-point	QUIC stack on both

Research Questions

RQ1: Which backend achieves the lowest migration latency?

RQ2: Does the choice of backend affect migration success rate?

RQ3: Can a network observer distinguish QUIC-based state transfer from regular QUIC traffic?

RQ4: Which backend is most resilient when the preferred server is not immediately ready?

RQ5: What is the operational overhead of each backend in a real deployment?

Task 2: QUIC Proxy — Pass-Through Handshake Exploration

> **Goal:** Explore whether the primary server can act as a proxy during the QUIC/TLS 1.3 handshake, forwarding handshake messages to the preferred server in real-time so it derives its own crypto keys. Identify the decryption problem. If a practical solution exists, compare latency against Task 1 baselines.

Motivation

In our current implementation, the primary completes the full TLS 1.3 handshake, then exports ~445 bytes of crypto state to the preferred server. This creates a window where the preferred has no crypto context. A pass-through model would let the preferred participate from the beginning, potentially eliminating the state transfer delay.

How It Would Work (Ideal Case)

- **Step 1:** Client sends Initial packet to the primary server.
- **Step 2:** Primary handles handshake normally BUT also forwards handshake packets to preferred in real-time.
- **Step 3:** Preferred silently processes the same handshake messages and derives identical TLS keys.
- **Step 4:** Handshake completes. Primary advertises `preferred_address`.
- **Step 5:** Client sends PATH_CHALLENGE to preferred.
- **Step 6:** Preferred already has the keys — responds immediately. Zero delay.

The Decryption Problem (Critical Challenge)

> **Key Issue:** Pure pass-through does NOT work due to TLS 1.3 ECDHE. The preferred server cannot derive traffic keys just by observing forwarded handshake packets. It lacks the primary's ephemeral private key needed for the Diffie-Hellman computation.

In TLS 1.3, the handshake uses ECDHE:

- Client sends ephemeral **PUBLIC** key in ClientHello.
- Primary generates ephemeral key pair, sends **PUBLIC** key in ServerHello.
- Both compute: `shared_secret = their_private × other_public`.

If primary forwards these packets, preferred sees both PUBLIC keys but does NOT have the primary's ephemeral PRIVATE key. Without it, it CANNOT compute the shared secret.

The primary would still need to send either the ephemeral private key OR the derived shared secret. This brings us back to crypto state export — just during the handshake instead of after. The latency benefit needs experimental verification.

Current vs. Pass-Through Comparison

Aspect	Current (Post-Handshake Export)	QUIC Proxy (Pass-Through)
When preferred gets keys	After handshake	During handshake (if feasible)

What is transferred	~445 bytes (secrets, CIDs, pkt#)	Ephemeral key or shared secret
Transfer delay	After handshake + transfer time	Overlaps with handshake
Decrypt issue?	No — full state exported	Yes — needs investigation
Complexity	Moderate	Higher (real-time fwd)

Research Questions

RQ1: Can we forward enough TLS state during the handshake for preferred to derive keys?

RQ2: Does forwarding the ephemeral key during handshake reduce latency vs. post-handshake export?

RQ3: What are the security implications? Does sharing ephemeral key weaken forward secrecy?

RQ4: Is there a way to involve preferred in key generation from the start (coordinated ECDHE)?

RQ5: Compatible with TLS 1.3 session resumption and 0-RTT?

RQ6: What is the minimum state that must be forwarded during handshake vs. after?

Deliverable

A feasibility report: (1) whether pass-through is possible given TLS 1.3, (2) decryption limitations, (3) if partial solution exists, (4) if feasible, latency benchmarks against Task 1 baselines.

Task 3: Applications of QUIC Migration

> **Goal:** Explore and research the real-world applications of QUIC server-side migration. Investigate use cases and content models, server type applicability, stream-level load balancing, and server-to-server chaining. Do basic research on each area and list what should be investigated further.

What is QUIC server migration actually useful for, and where does it fall short?

3a. Use Cases and Content Model

What are the concrete applications of QUIC server-side migration? For each, specify: server architecture, content model (same vs. different), migration workflow, and what additional state (beyond TLS) needs to migrate.

Same Content vs. Different Content

Scenario	Content Model	How It Works	Example
Horizontal LB (Replicas)	Same content on all servers	Client gets identical response from any	CDN edges, stateless APIs
Vertical Separation	Different roles, different content	Primary = gateway Preferred = backend	TLS terminator + app server
Stateful Services	Same content, shared state	App state in external store	Shopping cart, sessions
Malicious	Different (attacker ctrl)	Preferred serves attacker content	QUIC-Exfil

Application: Seamless Load Balancing (Horizontal)

Both servers are identical replicas. Primary handles TLS handshake, migrates to a less-loaded replica. After migration, replica communicates directly with client — no proxy in the data path. Key advantage over nginx: no proxy bottleneck.

Application: Vertical Separation

Primary is a lightweight TLS gateway. It handles handshake/auth, then migrates to a heavyweight application backend. Different roles, potentially different software. This is the 'vertical' split the professor mentioned.

Application: Zero-Downtime Maintenance

Active connections migrated to standby before primary goes offline. No client errors. Supports rolling updates.

Application: Geographic Optimization / Failover

Mid-session migration to better server. Sub-second failover without TCP reconnect. Not possible with DNS after connection established.

What State Needs to Migrate?

State Type	Currently Migrated?	Size	Needed For
TLS secrets	Yes	~128 bytes	All scenarios
Connection IDs	Yes	~40 bytes	All scenarios

Packet numbers	Yes	~16 bytes	All scenarios
HTTP/3 stream state	No	Variable	Mid-request migration
Application session	No	Variable	Stateful services
Request context (URL, headers)	No	Variable	Vertical separation
Flow control state	No	~32 bytes	High-throughput

What to Check Further

- For each application, what is the minimum state that must migrate?
- Can we define a generic 'application state export' API for different server types?
- Which application has the strongest case for a paper contribution?
- Are there real-world deployments we can reference or compare against?

3b. Server Type Applicability

Is QUIC migration equally applicable to all server types, or does each need separate implementation? Our 445-byte migration = TLS state only. What about stateful servers?

Server Type	Stateless?	App State to Migrate	Migration Complexity	Example
Static web	Yes	None	Easy — current impl sufficient	nginx HTML
REST API	Mostly	Session (maybe)	Easy if stateless	Microservices
Streaming (Netflix)	No	Stream position, buffer, bitrate	Hard — need playback state	Video, live
Database-backed	No	DB conn, transaction	Hard — DB don't migrate	E-commerce
WebSocket / real-time	No	Session, subscriptions	Hard — complex state	Chat, gaming
CDN edge	Yes (cache)	Cache (optional)	Easy if cached on both	Cloudflare

Key Insight

For **stateless servers** (static web, REST APIs, CDN), our current TLS-only migration is sufficient. For **stateful servers** (streaming, database, real-time), additional application state must transfer — our implementation doesn't handle this.

What to Check Further

- Can we define a generic 'application state export' interface for different server types?
- Is there a clean separation: transport migration (QUIC) vs. app migration (HTTP/3)?
- One framework or separate implementations per type?
- For Netflix-style streaming: what exactly needs to transfer? (bitrate, position, buffer)
- For database: can we migrate the QUIC connection but keep the DB connection separate?

3c. Stream-Level Load Balancing — Why It Doesn't Work

QUIC has independent multiplexed streams, which raises a natural question: could individual streams be routed to different servers? We analyze why this is not feasible with the current protocol.

Why Per-Stream Routing Breaks Down

Despite QUIC streams being logically independent, they share critical connection-level state that cannot be split:

- **Shared encryption:** All streams use the same TLS keys. A single QUIC packet can carry frames from multiple streams — you cannot decrypt one stream without decrypting the entire packet.
- **Shared packet numbers:** Packet numbers are per-connection, not per-stream. Multiple servers would need to coordinate packet number allocation to avoid collisions.
- **Shared congestion control:** QUIC runs one congestion controller per connection. Splitting streams across servers breaks congestion feedback.

• **Shared ACK state:** ACKs are per-connection. Server A cannot acknowledge packets that Server B received.

• RFC 9000 `preferred_address` migrates the entire connection, not individual streams. No per-stream mechanism exists.

Comparison: Split Streams vs. Separate Connections

Approach	Feasibility	Trade-off
Per-stream routing (one connection)	Not feasible with current QUIC	Would need new protocol, breaks crypto/congestion
Separate QUIC connections per service	Works today	More handshakes, but each connection is self-contained

What to Check Further

• Is there existing research on per-stream routing? If so, how do they handle shared state?

• Could a proxy model work where one server forwards specific stream frames to backends?

• Is the overhead of multiple separate connections actually a problem in practice?

3d. Server-to-Server Migration in Multi-Tier Architectures

In multi-tier architectures, a front-end server receives a client request and makes its own QUIC connection to a backend. If the backend advertises `preferred_address`, the front-end (acting as a QUIC client) follows the migration. This is a single-hop migration on the backend side, not a chain.

```
Client → Server A (front-end) → Server B (backend) → Server C (migrated backend)
```

The client's connection to A does not migrate. A's connection to B migrates to C. These are two independent connections, each with at most one migration.

Limitations

• `preferred_address` is a one-time handshake parameter. Once a connection migrates, it cannot migrate again — there is no mechanism for the preferred server to advertise a second preferred address.

• Cascading failover (B fails, so migrate to C) requires B to proactively migrate **before** failure. A dead server cannot export state. This is the same as zero-downtime maintenance (covered in 3a).

• Multi-hop relay (US → EU → Asia) would require repeated migrations on the same connection, which the protocol does not support.

What Is Feasible

• **Independent per-tier migration:** Client→A can migrate (client-facing tier). A→B can separately migrate (backend tier). Each tier manages its own connections.

• **Backend rebalancing:** Front-end's connection to backend B migrates to C. Front-end follows transparently. Useful for microservice rebalancing.

What to Check Further

• Is independent per-tier migration practical? What is the cumulative latency?

• Does backend migration add meaningful attack surface beyond single-hop?

• Are there real microservice architectures where backend QUIC migration helps?

Task 3 Summary: What to Explore

Sub-Task	Core Question	Check Further
3a. Use Cases & Content Model	What are the real uses? Same vs. different content?	Min state per app, generic API, paper angle
3b. Server Types	Works for all servers or just stateless?	App state interface, streaming/DB requirements
3c. Stream-Level LB	Why can't streams go to different servers?	Protocol constraints, separate connections as alternative
3d. Multi-Tier Migration	Can backend connections migrate independently?	Per-tier feasibility, microservice use cases

Task 4: Neqo Server & Server Architecture

> **Goal:** Investigate what Neqo's server actually supports and why someone said 'Neqo doesn't support server.' Compare QUIC migration architecture against traditional nginx reverse proxy. Understand what server architecture is needed to support migration and whether it can coexist with existing infrastructure.

4a. Neqo Server: What Does It Actually Support?

Someone said 'Neqo doesn't support server.' Neqo is built for Firefox (a client). The server component exists but may be limited to testing. We need to understand exactly what it can and cannot do, and whether it matters for our research.

What Neqo Server Has

- `neqo-bin/src/server/` — working QUIC server (HTTP/0.9 and HTTP/3 modes)
- `neqo-transport` with `Role::Server` — full QUIC transport layer
- `preferred_address` support (our modification)
- `Http3Server` in `neqo-http3`
- TLS certificate configuration via CLI

What May Be Missing

- Production concurrency (single-threaded event loop vs. multi-threaded workers?)
- Full HTTP/3 features (routing, virtual hosts, content negotiation?)
- Performance under load (max concurrent connections?)
- Graceful shutdown / connection draining
- Integration with existing infrastructure (nginx, health checks, monitoring)

Feature	Neqo Server	Production Server (nginx/quiche)	Impact on Research
QUIC transport	Full	Full	None
HTTP/3	Basic (test)	Full production	Limits app testing
preferred_address	Supported (our mod)	NOT implemented in production servers	This IS our contribution
Concurrency	Limited?	Thousands	Affects benchmarks
Content serving	Static/test	Full web server	Limits app testing
nginx integration	Unknown	Native	Needs investigation

What to Check Further

- Read neqo-bin/src/server/mod.rs — is it single-threaded?
- Run concurrent connection test: 10, 100, 1000 connections
- Compare Neqo server features against Cloudflare quiche and Google QUICHE
- Can Neqo work behind or alongside nginx?
- Does Neqo's limited server matter for our research, or is it sufficient for protocol testing?

4b. nginx & Server Architecture Comparison

nginx is the industry standard for load balancing. It supports QUIC/HTTP3 (since 1.25+) but does NOT implement preferred_address / server migration. Why not? What architecture is needed instead?

Architecture Comparison

nginx (proxy in every packet):

```
Client ↔ nginx ↔ Backend (nginx forwards ALL traffic, always in path)
```

QUIC migration (proxy exits after handshake):

```
Client ↔ Front-end (handshake) → migration → Backend (direct)
```

Dimension	nginx Reverse Proxy	QUIC Migration
Proxy in data path?	Yes, always	No — only handshake
Per-packet overhead	Every packet forwarded	Zero after migration
Server requirements	Only nginx needs QUIC	Both need QUIC stack
Failure handling	Retry to another backend	One-shot migration
Scalability ceiling	nginx is bottleneck	No bottleneck after
Health checks	Built-in	Not built-in
Routing logic	URL, headers, weight	Server decision at handshake only
Production ready?	Decades of use	Research (ours)

Why nginx Doesn't Support Migration

nginx's architecture is fundamentally a proxy — it sits in the middle and forwards packets. Server-side migration requires the proxy to **exit** the data path, which contradicts nginx's core design. Additionally, preferred_address requires sharing TLS secrets with the backend, which nginx's security model does not allow. The architecture needed for migration is fundamentally different from proxy.

What Server Architecture IS Needed?

- A front-end willing to give up its connection (not a persistent proxy)
- Backend servers with full QUIC+TLS capability (not just TCP backends)
- A secure state transfer channel between front-end and backend
- Coordination mechanism for load decisions at handshake time
- This is a fundamentally different architecture than traditional reverse proxy

What to Check Further

- Can QUIC migration work alongside nginx? (nginx as primary, QUIC backend as preferred)
- Is a hybrid model practical: nginx for routing + QUIC migration for handoff?
- At what traffic volume does proxy bottleneck matter vs. migration?
- What operational features (health checks, retries) does migration need to replicate?
- Would CDN providers benefit from migration as alternative to proxy?

Task 5: Attack Surface vs. Benefits — Tradeoff Analysis

Goal: Precisely define the attack surface opened by QUIC server-side migration and analyze the tradeoff: engineering advantages vs. security risks. Server-side migration is largely unimplemented specifically due to these concerns.

Why Server-Side Migration Is Rarely Implemented

Despite RFC 9000 Section 9.6, preferred_address is almost never implemented in production. Major implementations (Google QUICHE, Cloudflare quiche, nginx, msquic) all skip it. It opens an attack surface that didn't exist before.

Attack Surface Opened

- **Connection Hijacking:** Malicious server redirects client to attacker-controlled server. Client trusts preferred_address from TLS handshake — no independent verify.
- **Covert Exfiltration:** Preferred serves different content. QUIC-Exfil paper: ML classifiers achieve only 0-47% F1-score at detecting this.
- **State Transfer Interception:** 445 bytes of TLS secrets on the wire between servers. If intercepted, attacker decrypts all connection traffic.
- **Trust Boundary Expansion:** Every server receiving migration state = compromise point.
- **No Migration Attestation:** Client can choose not to use preferred_address (RFC 9000 §9.6: 'MAY'), but if it does, there is no mechanism to verify the preferred address belongs to the same legitimate operator. PATH_CHALLENGE validates reachability, not identity.
- **Content Substitution:** After migration, preferred controls what client receives.

Benefits Gained

- **No Proxy Bottleneck:** Front-end exits data path after migration. Direct client-server communication.
- **Sub-Second Failover:** Faster than TCP reconnect + TLS 1.3 re-handshake (2 RTTs).
- **Zero-Downtime Maintenance:** Drain connections gracefully before shutdown.
- **Vertical Separation:** Lightweight TLS terminator + heavyweight app server.
- **Geographic Optimization:** Mid-session migration to better server.

Risk/Benefit by Deployment Scenario

Scenario	Risk	Benefit	Verdict
Same-subnet replicas (stateless LB)	LOW	HIGH — no proxy bottleneck	Favorable
Vertical separation (same data center)	MEDIUM	HIGH — efficient resource usage	Likely favorable
Cross-network (WAN migration)	HIGH	MEDIUM — geo optimization	Needs safeguards
Cross-jurisdiction	VERY HIGH	LOW	Not recommended without attestation

Research Questions

RQ1: Minimum attacker capability to exploit server-side migration?

RQ2: Can attack surface be reduced without losing engineering benefits?

RQ3: Which applications have acceptable risk/benefit?

RQ4: Should RFC require client consent or migration attestation?

RQ5: Can we propose a migration_attestation transport parameter?

How the 5 Tasks Connect

- **Task 1 (Docker + Backends):** Benchmark 4 state transfer backends in Docker. Produces concrete latency and reliability numbers.
- **Task 2 (QUIC Proxy):** Investigate whether the preferred server can derive keys during the handshake instead of after. Document the ECDHE limitation.
- **Task 3 (Applications):** Define use cases, check which server types work, analyze why stream-level LB is not feasible, look at multi-tier scenarios.
- **Task 4 (Neqo + Architecture):** Check what Neqo's server actually supports. Compare migration architecture against nginx reverse proxy.
- **Task 5 (Attack Surface):** Document what attacks preferred_address enables and under what deployment conditions the risk is acceptable.